

Quest Planner Backend

Comprehensive Architectural Blueprint & Technical Manual: Deep-Dive into Every Class, Method, Security Layer, and Design Decision

SPRING BOOT 3.3.5 | JAVA 21 LTS | DUAL-TOKEN JWT & RBAC | POSTGRESQL JSONB

Author: Antigravity Full-Stack AI Pair Programmer

Project: Quest Planner (Full-Stack Visual Workspace)

Architecture Version: 2.0 (Short-Lived Access Token + Rotated Refresh Token + RBAC + In-Place Node Pruning)

Document Generated: October 2026

Scope: Exhaustive Codebase & Lifecycle Documentation

1. Executive Summary & Design Rationale

Quest Planner Backend is a cloud-ready, enterprise-grade REST API built using **Spring Boot 3.3.5** and **Java 21 LTS**. It serves as the authoritative persistence, security, and orchestration engine for the Quest Planner collaborative visual workspace.

1.1 Core Problems Solved & Key Design Choices

Challenge	Technical Implementation	Why We Did It (Rationale)
Dynamic Freeform Visual Data	PostgreSQL native `jsonb` column mapped with Hibernate 6 `@JdbcTypeCode(SqlTypes.JSON)` in `Board.java`.	A whiteboard canvas contains diverse, dynamically evolving visual primitives: To-Do lists with nested tasks, sticky notes with colors, images with 360° rotation and dimensions, and freehand pen vector strokes with point arrays. Normalizing this into 8 relational tables would cause crippling join overhead and rigid schema migrations. Storing it as a native JSONB column grants **sub-millisecond autosave** while retaining indexing and deep query capability.
Token Hijacking & Session Management	**Dual-Token Architecture**: Short-lived JWT Access Token (15 min) + Database-backed Long-lived Refresh Token (7 days) with **Refresh Token Rotation** and server-side revocation.	A single long-lived token cannot be revoked if stolen without invalidating all tokens globally. A single short-lived token forces frequent user re-login. Our dual-token design provides the gold standard: stateless authorization for ultra-fast API processing, paired with stateful refresh tokens in PostgreSQL that rotate on every use, enabling instant session revocation on logout.
Cross-User Data Isolation	Enforced scoping via `SecurityUtils.getCurrentUserId()` in `BoardServiceImpl` and `BoardRepository.findByIdAndUserId()`.	Prevent Insecure Direct Object References (IDOR). Even if a malicious user guesses another user's board ID, the repository query strictly binds to the authenticated user ID extracted from the verified JWT, returning `404 Not Found` rather than leaking unauthorized board data.
Administrator Oversight vs. User Privacy	**Strict Privacy Guard**: Admin endpoints only run aggregate count queries (`countByUserId`).	Admins need high-level metrics (total users, joined dates, total plan count per user) to manage platform health. However, users' brainstorms, sprint plans, sketches, and images must remain strictly private. The backend architecture guarantees that no canvas data, board titles, or notes are ever loaded or sent to admin accounts.
Privilege Escalation Protection	Hardcoded `Role.ROLE_USER` on account registration; admin status can only be elevated directly via PostgreSQL SQL update.	Public registration endpoints must never allow users to declare their own privileges. Restricting admin elevation to direct database operations guarantees zero unauthorized privilege escalation from the web layer.
Atomic Node Deletion with Edge Cascading	Custom in-memory JSONB filter in `deleteNode(boardId, nodeId)` removing the node and any attached edges where `source == nodeId || target == nodeId`.	Ensures graph integrity. Deleting a card in React Flow without cascading would leave orphan dangling edges pointing to nonexistent nodes, breaking canvas rendering and graph traversal.

2. Complete Package & Class Reference

Every single class, interface, and helper in the backend is detailed below with its specific purpose, design reasoning, and method breakdown.

2.1 Application Entry Point

com.whiteboard.backend.BackendApplication

@SPRINGBOOTAPPLICATION

Purpose: Serves as the primary bootstrap entry point for the Spring Boot application.

> **`public static void main(String[] args)`**
> Invokes `SpringApplication.run(BackendApplication.class, args)` to initialize the Spring ApplicationContext, start the embedded Tomcat servlet engine on the configured port (default 8080 or `$PORT` on Render), and trigger auto-configuration for JPA, Security, and WebMvc.

2.2 Configuration Package (`config`)

CorsConfig.java

@CONFIGURATION

Purpose: Configures Cross-Origin Resource Sharing (CORS) rules to allow the Vite development server and production Vercel frontend domains to communicate with the REST API.

Fields:

- `allowedOrigins`: Injected from `${cors.allowed-origins}` (e.g. `http://localhost:5173,http://localhost:3000`).
- `allowedOriginPatterns`: Injected from `${cors.allowed-origin-patterns}` (e.g. `https://*.vercel.app`) to dynamically allow all preview and production deployment URLs on Vercel.
- `allowedMethods`: `GET,POST,PUT,DELETE,OPTIONS`.
- `allowedHeaders`: `"*"` (allows Authorization, Content-Type, etc.).
- `maxAge`: `3600` seconds (1 hour preflight cache to minimize preflight roundtrips).

> **`public WebMvcConfigurer corsConfigurer()`**
> Registers CORS mappings for `/api/**` endpoints with credentials enabled (`allowCredentials(true)`) and wildcard origin pattern matching for seamless Vercel preview environments.

SecurityConfig.java

@CONFIGURATION @ENABLEWEBSECURITY @ENABLEMETHODSECURITY

Purpose: Configures Spring Security 6 filter chains, stateless session management, BCrypt password hashing, and endpoint authorization rules.

`public PasswordEncoder passwordEncoder()`

Returns a `BCryptPasswordEncoder` instance with standard work factor (10 rounds). Used to salt and hash user passwords before database storage and to securely verify login passwords.

`public AuthenticationManager authenticationManager(AuthenticationConfiguration authConfig)`

Exposes the Spring Security `AuthenticationManager` bean from the active configuration.

`public SecurityFilterChain securityFilterChain(HttpSecurity http)`

Builds the HTTP security filter chain:

- Disables CSRF (safe because authentication is fully stateless via JWT Bearer headers, eliminating browser cookie vulnerability).
- Integrates CORS filter with Spring Security.
- Sets session creation policy to `SessionCreationPolicy.STATELESS` (no HTTP sessions created or stored).
- Configures custom `JwtAuthenticationEntryPoint` to return structured JSON on 401s.
- Authorizes preflight `OPTIONS` requests and public endpoints (`/api/v1/auth/**` , `/api/health`).
- Secures `/api/v1/admin/**` with `hasAuthority('ROLE_ADMIN')` .
- Requires authentication for `/api/v1/boards/**` and all remaining endpoints.
- Inserts `JwtAuthenticationFilter` before `UsernamePasswordAuthenticationFilter` .

2.3 Data Entity Models (`entity`)

User.java

`@ENTITY @TABLE(NAME="USERS")`

Purpose: Represents a registered user in the database with credentials, role, and audit timestamp.

Fields:

- `Long id`: Primary key, auto-incrementing identity strategy.
- `String username`: Unique, non-null, max 100 characters. Indexed via `idx_users_username`.
- `String email`: Unique, non-null, max 255 characters. Indexed via `idx_users_email`.
- `String password`: BCrypt hashed password string. Never stored in plaintext.
- `Role role`: Enum stored as String (`ROLE_USER` or `ROLE_ADMIN`).
- `LocalDateTime createdAt`: Timestamp set automatically on record creation.

> **`protected void onCreate()`**
> `@PrePersist` callback that automatically stamps `createdAt = LocalDateTime.now()` before insertion.

Board.java

`@ENTITY @TABLE(NAME="BOARDS")`

Purpose: Represents a whiteboard planning workspace containing title, description, user ownership relation, and the complete canvas state JSON.

Fields:

- `Long id`: Primary key.
- `String title`: Required human-readable name of the board (up to 255 chars).
- `String description`: Optional markdown/text summary.
- `User user`: `@ManyToOne(fetch = FetchType.LAZY)` foreign key linking the board to its owning `User`.
- `Map<String, Object> canvasData`: Native PostgreSQL `jsonb` column mapped with Hibernate `@JdbcTypeCode(SqlTypes.JSON)`. Stores the full graph structure (nodes array, edges array, vector drawings array, viewport zoom/pan).
- `LocalDateTime createdAt` & `LocalDateTime updatedAt`: Audit timestamps.

> **`protected void onCreate() & protected void onUpdate()`**
> Lifecycle callbacks setting timestamps on initial persist and updating `updatedAt` on every modification.

RefreshToken.java

@ENTITY @TABLE(NAME="REFRESH_TOKENS")

Purpose: Stores long-lived session renewal tokens linked to users, supporting token rotation and server-side revocation.

Fields:

- `Long id`: Primary key.
- `User user`: `@ManyToOne(fetch = FetchType.LAZY)` linking the refresh token to the user account.
- `String token`: Unique cryptographically random opaque token string (64 characters).
- `Instant expiryDate`: Exact UTC expiration timestamp (set to 7 days from generation).
- `boolean revoked`: Flag indicating if the token was invalidated prior to expiration.
- `Instant createdAt`: Audit timestamp.

Role.java

ENUM

Values: `ROLE_USER` (standard user), `ROLE_ADMIN` (system administrator with oversight access).

2.4 Data Repositories (`repository`)

UserRepository.java

@REPOSITORY JPAREPOSITORY<USER, LONG>

Methods:

`Optional<User> findByUsername(String username)`

Looks up user by exact username for authentication.

`Optional<User> findByEmail(String email)`

Looks up user by normalized email.

`Optional<User> findByUsernameOrEmail(String username, String email)`

Allows users to log in using either their username or email address seamlessly.

`boolean existsByUsername(String username) & boolean existsByEmail(String email)`

Fast existence checks during registration to prevent duplicate accounts.

BoardRepository.java

@REPOSITORY JPAREPOSITORY<BOARD, LONG>

Methods:

`List<Board> findAllByUserIdOrderByUpdatedAtDesc(Long userId)`

Fetches boards belonging strictly to the specified user, sorted by most recently modified first.

`Optional<Board> findByIdAndUserId(Long id, Long userId)`

Guarantees user isolation: loads a board by ID only if it is owned by the specified user. Prevents IDOR attacks.

`long countByUserId(Long userId)`

Executes an ultra-fast `SELECT COUNT(*) FROM boards WHERE user_id = ?` for the Admin Oversight Dashboard without loading board contents.

RefreshTokenRepository.java

@REPOSITORY JPAREPOSITORY<REFRESHTOKEN, LONG>

Methods:

`Optional<RefreshToken> findByToken(String token)`

Retrieves stored refresh token record by token string.

`@Modifying void deleteByUser(User user)`

Deletes all previous refresh tokens for a user when establishing a fresh login session.

`@Modifying void deleteByToken(String token)`

Revokes and deletes a specific refresh token upon explicit user logout.

2.5 Security Engine & JWT Infrastructure (`security`)

JwtTokenProvider.java

@COMPONENT

Purpose: Central engine for generating, signing, and validating short-lived JWT access tokens and long-lived refresh tokens using the modern JJWT 0.12+ API.

`private SecretKey getSigningKey()`

Decodes the Base64/UTF-8 configured secret key and returns an HMAC-SHA `SecretKey` guaranteed to meet the 256-bit cryptographic requirement.

`public String generateAccessToken(Long userId, String username, String role)`

Generates a signed JWT with 15-minute expiration (`jwt.expiration-ms: 900000`). Injects subject (username), `userId` claim, `role` claim, and `type: "ACCESS"` claim.

`public String generateRefreshTokenString()`

Generates a high-entropy 64-character alphanumeric opaque token string using two combined UUIDs with hyphen removal.

`public String getUsernameFromToken(String token)`

Parses the signed JWT and extracts the subject (username).

`public Long getUserIdFromToken(String token)`

Extracts the `userId` claim safely, handling both Long and Number types.

`public String getRoleFromToken(String token)`

Extracts the `role` claim string (e.g. `ROLE_USER`).

`public boolean validateToken(String token)`

Validates signature, structure, and expiration date using `Jwts.parser().verifyWith(key).build().parseSignedClaims(token)` . Catches ExpiredJwtException, MalformedJwtException, and SecurityException.

JwtAuthenticationFilter.java

@COMPONENT ONCEPERREQUESTFILTER

Purpose: HTTP filter that intercepts incoming requests, extracts the JWT Bearer token, validates it, and establishes the authenticated security context.

`protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)`

Extracts token from `Authorization: Bearer <token>` header. If present and valid, retrieves username, loads `UserDetails` via `CustomUserDetailsService` , and injects a `UsernamePasswordAuthenticationToken` into `SecurityContextHolder.getContext()` .

`private String getJwtFromRequest(HttpServletRequest request)`

Helper that verifies header begins with "Bearer " and extracts the substring token.

SecurityUtils.java

UTILITY CLASS

Purpose: Provides static convenience methods to query the authenticated user without boilerplate.

> **`public static Long getCurrentUserId()`**
>
> Extracts the authenticated user ID from `SecurityContextHolder`. Used by all services to scope queries to the caller.

> **`public static String getCurrentUsername()`**
>
> Extracts the authenticated username.

JwtAuthenticationEntryPoint.java

@COMPONENT AUTHENTICATIONENTRYPOINT

Purpose: Intercepts unauthenticated requests that fail authorization, returning an HTTP 401 with a standardized JSON error body matching `ErrorResponse` rather than default HTML.

2.6 Business Logic & Services (`service`)

AuthServiceImpl.java

@SERVICE @TRANSACTIONAL

Purpose: Handles user registration, credential authentication, refresh token rotation, and logout session termination.

`public AuthResponse register(RegisterRequest request)`

1. Validates that username and email are not already registered (throws IllegalArgumentException if taken).
2. Enforces standard role assignment: `Role.ROLE_USER` (DB-only admin provisioning).
3. Hashes password using BCrypt.
4. Saves `User` record.
5. Generates short-lived 15-minute `accessToken`.
6. Creates and persists a 7-day `RefreshToken` in the database.
7. Returns `AuthResponse` containing both tokens and user profile.

`public AuthResponse login(LoginRequest request)`

1. Looks up user by username or email.
2. Verifies password match via `passwordEncoder.matches()` (throws BadCredentialsException if invalid).
3. Clears old refresh tokens for this user via `refreshTokenRepository.deleteByUser(user)`.
4. Issues new 15-minute access token and new 7-day refresh token.
5. Returns full `AuthResponse`.

`public AuthResponse refreshToken(RefreshTokenRequest request)`

1. Retrieves refresh token from DB.
2. Validates that token is not revoked and `expiryDate.isAfter(Instant.now())`.
3. If expired or revoked, deletes token and throws BadCredentialsException.
4. **Refresh Token Rotation**: Updates the existing token string with a newly generated opaque token and extends the expiry by 7 days.
5. Issues a new 15-minute access token.
6. Returns updated `AuthResponse`.

`public void logout(String refreshToken)`

If a refresh token is supplied, deletes it from the database so the session can never be renewed.

BoardServiceImpl.java

@SERVICE @TRANSACTIONAL

Purpose: Full CRUD operations, autosave serialization, user scoping, and in-place node deletion.

`public List<BoardSummaryResponse> getAllBoards()`

Retrieves all boards owned by the authenticated caller. Maps each entity to lightweight `BoardSummaryResponse` (omitting canvas data to keep dashboard payloads tiny).

`public BoardResponse getBoardById(Long id)`

Loads the full board details including complete JSONB canvas data via `findByIdAndUserId(id, userId)`.

`public BoardResponse createBoard(CreateBoardRequest request)`

Initializes an empty canvas map containing `nodes: []` and `edges: []`, binds it to the current user, saves, and returns `BoardResponse`.

`public BoardResponse updateBoard(Long id, UpdateBoardRequest request)`

Supports partial updates (autosave): overwrites title, description, and/or canvasData if non-null in payload. Updates `updatedAt` automatically.

`public void deleteBoard(Long id)`

Permanently deletes the board entity owned by the user.

`public BoardResponse deleteNode(Long boardId, String nodeId)`

1. Loads board ensuring caller ownership.
2. Retrieves `canvasData` map.
3. Filters the `nodes` array to drop the element matching `nodeId`.
4. **Cascade Edge Removal**: Filters the `edges` array to drop any edge where `source == nodeId || target == nodeId`.
5. Saves the updated board and returns full response.

AdminServiceImpl.java

@SERVICE @TRANSACTIONAL(READONLY=TRUE)

Purpose: Implements the platform oversight query with strict privacy guard.

`public List<AdminUserSummaryResponse> getAllUsersSummary()`

Queries all users ordered by creation date. For each user, computes total plan count via `boardRepository.countByUserId(userId)`. Never touches or leaks board titles, notes, or canvas content.

2.7 REST Controllers (`controller`)

AuthController.java

`@RestController @RequestMapping("/api/v1/auth")`

Endpoints:

- `POST /register`: Takes `RegisterRequest`, returns 201 Created with `AuthResponse`.
- `POST /login`: Takes `LoginRequest`, returns 200 OK with `AuthResponse`.
- `POST /refresh`: Takes `RefreshTokenRequest`, returns 200 OK with rotated `AuthResponse`.
- `POST /logout`: Takes optional `RefreshTokenRequest`, revokes session, returns 200 OK with message.
- `GET /health`: Health check endpoint returning status UP.

BoardController.java

`@RestController @RequestMapping("/api/v1/boards")`

Endpoints:

- `GET /api/v1/boards`: List summaries of boards owned by user.
- `GET /api/v1/boards/{id}`: Full board details with canvas JSON.
- `POST /api/v1/boards`: Create new board (title & optional description).
- `PUT /api/v1/boards/{id}`: Autosave / update canvas data and title.
- `DELETE /api/v1/boards/{id}`: Delete board permanently.
- `DELETE /api/v1/boards/{id}/nodes/{nodeId}`: Delete specific node and prune edges.

AdminController.java

`@RestController @RequestMapping("/api/v1/admin")`

Endpoints:

- `GET /api/v1/admin/users`: Restricted to `ROLE_ADMIN` via `@PreAuthorize("hasAuthority('ROLE_ADMIN')")`. Returns user oversight metrics.

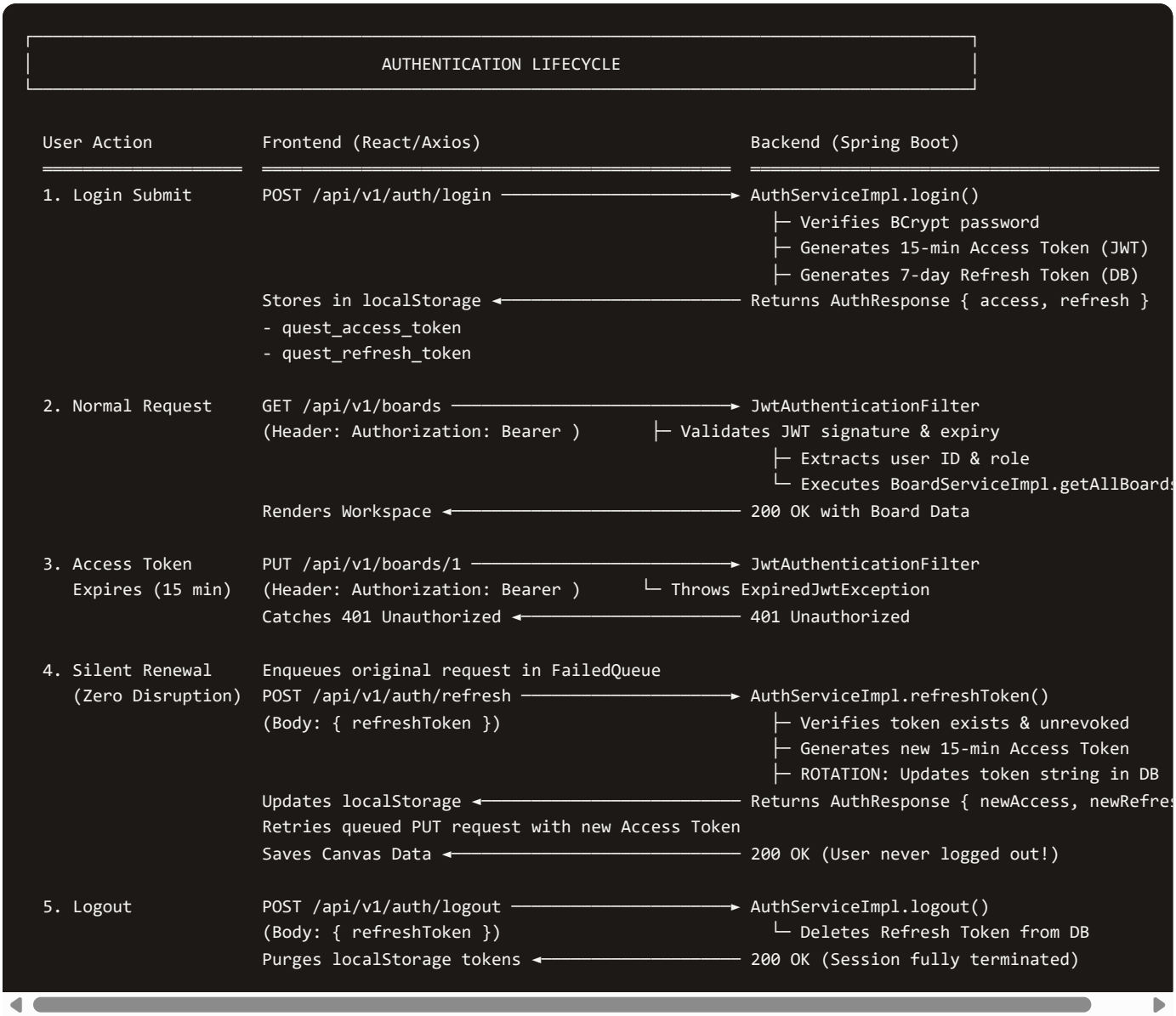
2.8 Data Transfer Objects & Exception Handling

DTO / Class	Package	Key Fields & Validation	Role in System
`RegisterRequest`	dto.request	`username` (@NotBlank, max 100), `email` (@NotBlank, @Email), `password` (@NotBlank, min 6)	Validated signup payload.
`LoginRequest`	dto.request	`username` (@NotBlank), `password` (@NotBlank)	Credential submission for login.
`RefreshTokenRequest`	dto.request	`refreshToken` (@NotBlank)	Used to renew access tokens or logout.
`CreateBoardRequest`	dto.request	`title` (@NotBlank, max 255), `description`	Board initialization payload.
`UpdateBoardRequest`	dto.request	`title`, `description`, `canvasData` (Map)	Autosave payload for canvas modifications.
`AuthResponse`	dto.response	`accessToken`, `refreshToken`, `tokenType`, `expiresIn`, `id`, `username`, `email`, `role`	Unified auth response. Also includes backward-compatible `getToken()` alias.
`BoardResponse`	dto.response	`id`, `title`, `description`, `canvasData`, `createdAt`, `updatedAt`	Full board response with complete visual tree.
`BoardSummaryResponse`	dto.response	`id`, `title`, `description`, `createdAt`, `updatedAt`	Dashboard listing payload (canvas data omitted).
`AdminUserSummaryResponse`	dto.response	`id`, `username`, `email`, `role`, `createdAt`, `planCount`	Privacy-guarded admin oversight record.
`ErrorResponse`	dto.response	`timestamp`, `status`, `message`, `path`	Standardized error output for all API errors.
`GlobalExceptionHandler`	exception	Centralized `@RestControllerAdvice`	Catches ResourceNotFoundException (404), MethodArgumentNotValidException (400), BadCredentialsException (401), AccessDeniedException (403), and unexpected runtime errors (500).

3. Security & Dual-Token Lifecycle Deep Dive

Below is the complete end-to-end trace of how the Access Token and Refresh Token cooperate between Spring Boot and the React frontend during active usage:

3.1 Token Lifecycle Flowchart



3.2 Why Refresh Token Rotation Matters

In a standard single-token or non-rotated architecture, if an attacker intercepts a long-lived refresh token, they retain access for the entire validity period. With **Refresh Token Rotation**, every time a token is exchanged for a new access token, the refresh token itself is invalidated and replaced with a new one. If an attacker attempts to reuse an old token, the server detects the token is already used or missing, revokes the user's sessions, and forces credential re-authentication.

4. Production Configuration & Cloud Deployment

4.1 Environment Variable Mapping

Variable	Default Value	Purpose & Target Platform
`PORT`	`8080`	Server port. Injected automatically by Render (usually port 10000).
`SPRING_DATASOURCE_URL`	`jdbc:postgresql://localhost:5432/whiteboard_db`	Database JDBC URL. Set to Neon PostgreSQL host with `sslmode=require` in production.
`SPRING_DATASOURCE_USERNAME`	`postgres`	PostgreSQL database username.
`SPRING_DATASOURCE_PASSWORD`	`postgres`	PostgreSQL database password.
`JWT_SECRET`	`404E63...`	256-bit cryptographic signing key for HMAC-SHA256 tokens.
`JWT_EXPIRATION_MS`	`900000` (15 mins)	Short-lived access token validity duration.
`JWT_REFRESH_EXPIRATION_MS`	`604800000` (7 days)	Long-lived refresh token validity duration.
`CORS_ALLOWED_ORIGIN_PATTERNS`	`https://*.vercel.app`	Permits all Vercel production and staging preview deployments to call the API.

4.2 Docker Multi-Stage Build (Dockerfile on Render)

The backend includes an optimized multi-stage `Dockerfile` located at `backend/Dockerfile`:

- **Stage 1 (Build)**: Uses `maven:3.9.9-eclipse-temurin-21-alpine` to compile the Java source and package the JAR with dependencies skipped during testing for fast container builds.
- **Stage 2 (Runtime)**: Uses lightweight `eclipse-temurin:21-jre-alpine`, copying only the final 60MB executable fat JAR. This produces a minimal container image, fast boot times, and tiny attack surface.

> **Conclusion:** The Quest Planner backend is robust, strictly isolated, privacy-compliant, and fully resilient against token hijacking and session expiration.